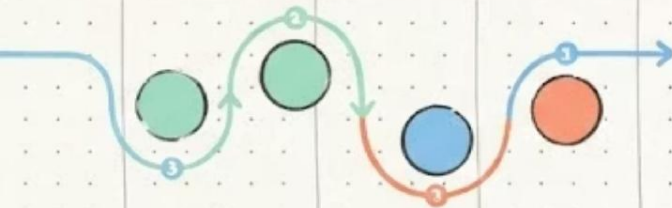


Gestão do Ciclo de Vida da Aplicação

Design Patterns (Parte 1)

Professor Romário da Silva Borges

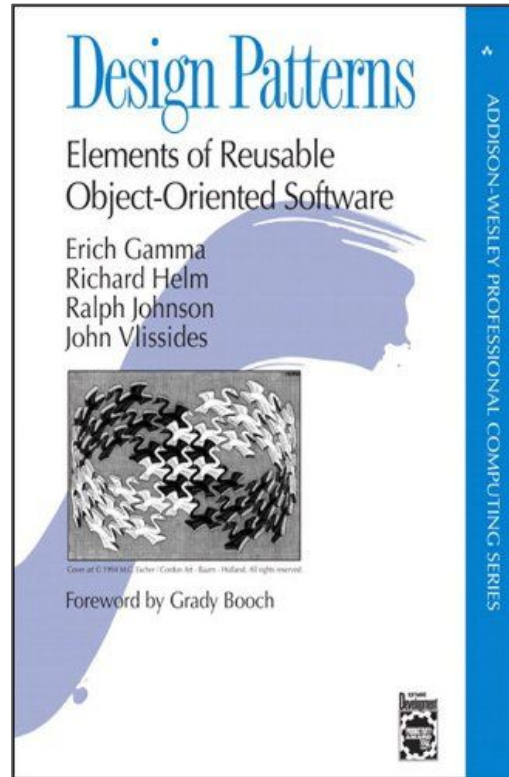


Utilidade #1: Reúso de projeto

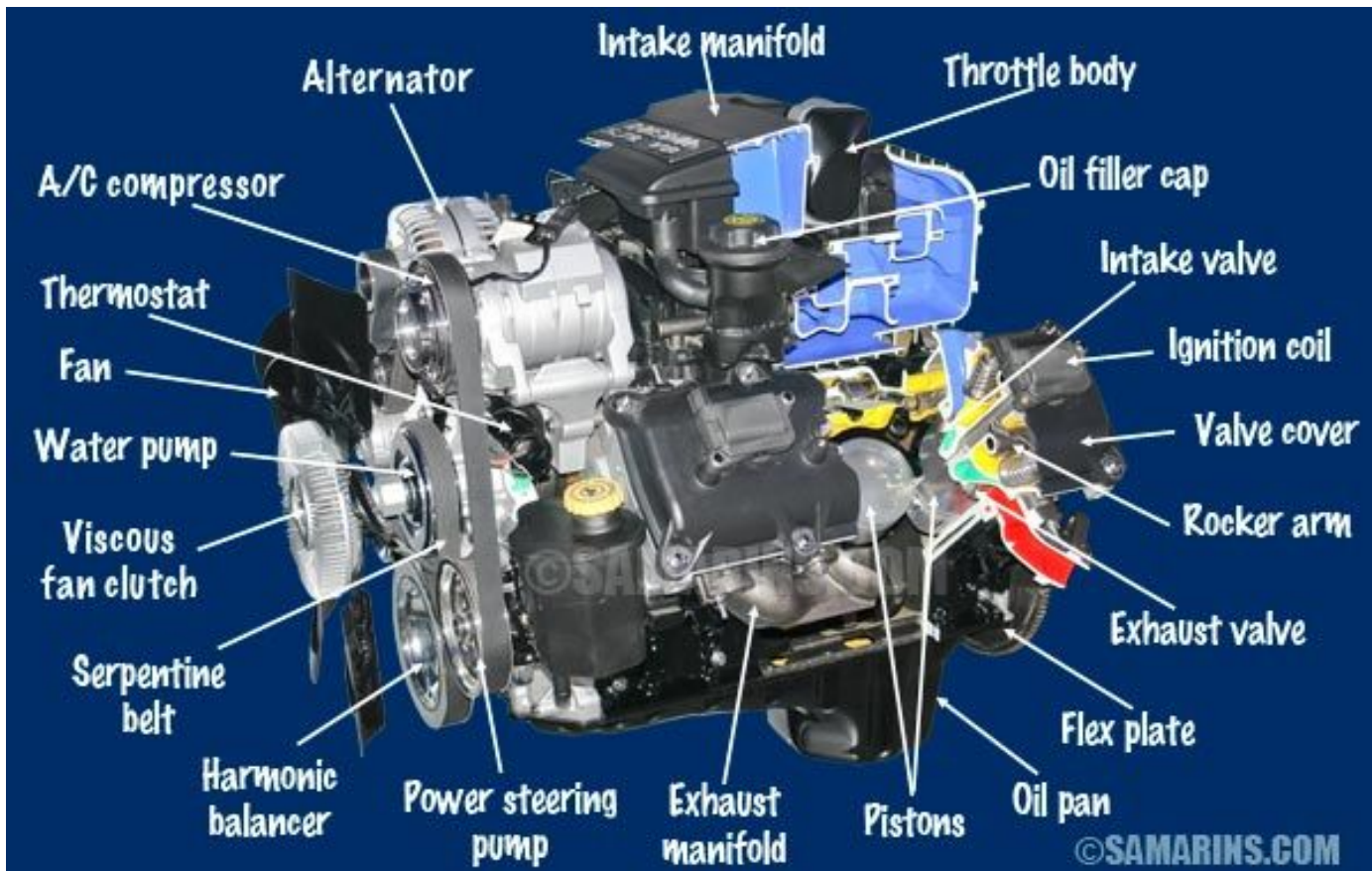
- Suponha que eu tenho um problema de projeto
- Pode existir um padrão que resolve esse problema
- Vantagem: ganho tempo e não preciso reinventar a roda

Utilidade #2: Vocabulário para comunicação

- Vocabulário para discussões, documentação, etc



1994, conhecido como livro da
"Gangue dos Quatro" ou GoF



Quais são as "peças" que posso usar no design do meu software?

Criacionais	Estruturais	Comportamentais
Abstract Factory (6.2) Factory Method Singleton (6.3) Builder (6.12) Prototype	Proxy (6.4) Adapter (6.5) Facade (6.6) Decorator (6.7) Bridge Composite Flyweight	Strategy (6.8) Observer (6.9) Template Method (6.10) Visitor (6.11) Chain of Responsibility Command Interpreter Iterator (6.12) Mediator Memento State

23 padrões

Padrões de Projeto

- Soluções recorrentes para problemas de projeto enfrentados por engenheiros de software

Estrutura da Apresentação dos Padrões

- Contexto (sistema ou parte de um sistema)
- Problema (de projeto)
- Solução (por meio de um padrão de projeto)

Importante

- Padrões de projeto ajudam em design for change
- Facilitam mudanças futuras no código
- Se o código dificilmente vai precisar de mudar, então uso de padrões é um exemplo de overengineering

Complementando a comparação anterior:

Quais são as "peças" que posso usar no design do meu software para torná-lo flexível e fácil de modificar?

Criacionais

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves flow from the bottom left towards the right side of the frame. Interspersed within these waves are faint, semi-transparent snippets of code, likely from a programming language like JavaScript or PHP, which are partially obscured by the motion of the waves.

Os padrões criacionais compreendem padrões que propõem soluções flexíveis para criação de objetos. São eles: Abstract Factory, Factory Method, Singleton, Builder e Prototype.

1 - Fábrica

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves flow from the left side towards the right. Interspersed within these waves are faint, semi-transparent snippets of code, likely in JavaScript or a similar language, which appear to be part of a web application's logic, such as form validation or data handling. The overall aesthetic is clean, modern, and tech-oriented.

Relembrando a estrutura da explicação

- Contexto
- Problema
- Solução

Contexto: Sistema que usa canais de comunicação

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```


Problema

- Alguns clientes vão precisar de usar UDP, em vez de TCP
- Como parametrizar as chamadas de **new**?
- Estão espalhadas em vários pontos do código
- Como criar um projeto que facilite essa mudança?

Solução: Padrão Fábrica

- Fábrica: método que centraliza a criação de certos objetos

```
class ChannelFactory {  
    public static Channel create() { // método fábrica estático  
        return new TCPChannel();  
    }  
}
```

Único ponto de mudança se quisermos mudar para UDP

Sem Fábrica

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```

Sem Fábrica

```
void f() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void g() {  
    TCPChannel c = new TCPChannel();  
    ...  
}  
  
void h() {  
    TCPChannel c = new TCPChannel();  
    ...  
}
```



Com Método Fábrica Estático

```
void f() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void g() {  
    Channel c = ChannelFactory.create();  
    ...  
}  
  
void h() {  
    Channel c = ChannelFactory.create();  
    ...  
}
```

2 - Singleton

The background of the slide features a light blue and white gradient. Overlaid on this are several wavy, horizontal lines in shades of purple and blue. These lines appear to be composed of or contain snippets of code, likely in C++, which are blurred and partially obscured by the waves. The overall aesthetic is clean and modern, typical of a technical presentation.

```
void f() {  
    Logger log = new Logger();  
    log.println("Executando f");  
    ...  
}  
void g() {  
    Logger log = new Logger();  
    log.println("Executando g");  
    ...  
}  
void h() {  
    Logger log = new Logger();  
    log.println("Executando h");  
    ...  
}
```

Contexto:
Classe Logger

```
void f() {  
    Logger log = new Logger();  
    log.println("Executando f");  
    ...  
}  
void g() {  
    Logger log = new Logger();  
    log.println("Executando g");  
    ...  
}  
void h() {  
    Logger log = new Logger();  
    log.println("Executando h");  
    ...  
}
```

Problema:

Cada método cliente
usa sua própria
instância de Logger

Problema

- Como fazer com que todos os clientes usem a mesma instância de Logger?
- Na verdade, deve existir uma única instância de Logger
- Todo "log" seria registrado no mesmo arquivo

Solução: **Padrão Singleton**

- Transformar a classe Logger em um Singleton
- Singleton: classe que possui no máximo uma instância

```
class Logger {  
    private Logger() {} // proíbe clientes chamar new Logger()  
  
    private static Logger instance; // instância única da classe  
  
    public static Logger getInstance() {  
        if (instance == null) // 1a vez que chama-se getInstance  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // registra msg na console, mas poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```

```
class Logger {  
  
    private Logger() {} // proíbe clientes chamar new Logger()  
  
    private static Logger instance; // instância única da classe  
  
    public static Logger getInstance() {  
        if (instance == null) // 1a vez que chama-se getInstance  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // registra msg na console, mas poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```

```
class Logger {  
  
    private Logger() {} // proíbe clientes chamar new Logger()  
  
    private static Logger instance; // instância única da classe  
  
    public static Logger getInstance() {  
        1 if (instance == null) // 1a vez que chama-se getInstance  
            instance = new Logger();  
        2 return instance;  
    }  
  
    public void println(String msg) {  
        // registra msg na console, mas poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```

```
class Logger {  
  
    private Logger() {} // proíbe clientes chamar new Logger()  
  
    private static Logger instance; // instância única da classe  
  
    public static Logger getInstance() {  
        if (instance == null) // 1a vez que chama-se getInstance  
            instance = new Logger();  
        return instance;  
    }  
  
    public void println(String msg) {  
        // registra msg na console, mas poderia ser em arquivo  
        System.out.println(msg);  
    }  
}
```

```
void g() {  
    Logger log = Logger.getInstance();  
    log.println("Executando g");  
    ...  
}  
  
void h() {  
    Logger log = Logger.getInstance();  
    log.println("Executando h");  
    ...  
}
```

Mesma
instância

Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: <https://engsoftmoderna.info>.

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

